

USER MANUAL

HexLogic (8051 + ARM) Web Emulator

This manual explains how to use HexLogic’s browser-based emulator UI, how the simulation works behind the scenes, how to use the Virtual Hardware board (LEDs, switches, etc.), and what to do when something doesn’t work.

1. Introduction

HexLogic is a **web-based assembly IDE + emulator**. It lets you:

- Write assembly code (primarily **8051 / AT89C51-style**).
- Assemble it into machine code.
- Run or single-step the program.
- Observe CPU state (registers, flags, call stack, trace).
- Inspect memory and code ROM.
- Use a **Virtual Hardware** board to connect simulated pins to devices (LED, switch, LED array, 7-segment, stepper).

It also includes a **minimal ARM sandbox** (not cycle-accurate) for basic instruction execution and GPIO-style MMIO.

2. System Overview

2.1 Purpose of the application

HexLogic is designed for learning and debugging low-level programs by making CPU state changes visible while you run assembly code.

2.2 Major modules (what exists in the codebase)

Frontend (UI)

- Templates + layout: `./api/templates/index.html`, `./api/templates/_app_shell.html`
- UI logic (buttons, rendering, editor, hardware UI): `./api/static/sim8051-app.js`
- REST client wrapper: `./api/static/sim8051-client.js`
- Styling: `./api/static/styles.css`
- Monaco editor loader (via CDN): included in `./api/templates/index.html`

Backend API (Flask)

- Flask entry point: `./api/index.py`
- REST API (v2): `./api/sandbox_api.py`
- Session cookie: `hexlogic_session` (browser cookie)

Simulation / Emulator Core

- Session orchestration: `./sim8051/session.py`
- CPU models: `./sim8051/cpu.py` (8051), `./sim8051/cpu_arm.py` (ARM)
- Memory model + GPIO port modeling: `./sim8051/memory.py`
- Assembler(s): `./sim8051/assembler.py`, `./sim8051/assembler_arm.py`
- Debugger model (breakpoints/watchpoints/trace/history): `./sim8051/base_cpu.py`, `./sim8051/model.py`

Virtual Hardware / IO Simulation

- Hardware devices + wiring + validation + faults: `./sim8051/hardware.py`

2.3 How components interact (data flow)

At a high level:

1. **You** click a UI control (Assemble/Run/Step/Hardware actions).
2. The **frontend** calls the backend using JSON REST calls to `/api/v2/*`.
3. The **backend** routes the request to a per-user **SimulatorSession**.
4. The **session** executes CPU steps, updates memory, runs debugger rules (breakpoints), and synchronizes the **VirtualHardwareManager**.
5. The backend returns a **snapshot** (and sometimes a **diff**) of state.
6. The frontend re-renders panels (registers, memory, trace, hardware board) and updates editor highlighting.

The frontend also opens a server-sent events stream:

- `GET /api/v2/events/signals` to receive live hardware signal updates during running execution (if the browser supports `EventSource`).

3. Interface Layout (with section descriptions)

3.1 Title bar (top)

Location: very top of the page **What you see:**

- HexLogic logo + name
- "Target: 8051" chip indicator (updates when architecture changes)
- "Sandboxed Multi-Architecture Debug View" label

3.2 Main toolbar (controls row)

Location: directly under the title bar **What it contains:** run controls, view toggles, architecture settings, speed, base converter, export/import, waveform, help, theme, breakpoint clearing, memory edit.

All toolbar elements are documented in Section 4.

3.3 Workspace (main area)

The workspace is a 3-column IDE layout:

Left pane

- **Target Explorer:** a simple tree (informational)
- **Breakpoints summary:** list of active breakpoint line numbers
- **Execution State:** architecture, PC, cycles, etc.
- **Registers and Flags:** current register values + flags + timer snapshot (when available)
- **Call Stack:** return address stack from calls/returns (debugger view)

Center pane

- **Editor (Monaco):** your assembly source; click gutter to toggle breakpoints
- **Assembler Output:** listing table mapping source lines → addresses → bytes
- **Debugger Console:** textual log of actions/errors/run summaries
- **Trace Timeline:** recent executed instructions + register diffs

Right pane

- **Memory:** Internal RAM (or "Register Shadow" for ARM), XRAM sample (or "Data Memory" for ARM), Code ROM
- **Runtime Metrics:** cycles, effective throughput, UI timing statistics, and backend metrics

3.4 Status bar (bottom)

Location: bottom row **Elements:**

- Left: status text (e.g., "Ready.", "Running...", error messages)

- Right: mode line (architecture, PC, cycles)

3.5 Waveform drawer (optional)

Location: overlays as a right-side drawer when opened **Purpose:** quick "logic waveform" previews from recent digital signal activity detected via virtual hardware signal logging.

3.6 Help popover (optional)

Location: near the toolbar Help button **Content:** contact email link

3.7 Virtual Hardware workspace (optional)

Activated by switching **View → Hardware**.

It provides:

- Component palette (LED, Switch, LED Array, 7-Segment, Stepper)
- A board canvas with an MCU pin map and draggable devices
- Wire drawing (pin ↔ device)
- A debug panel: port monitor, component state, validation issues, signal log, logic analyzer, inspector, test results

4. Controls & Buttons (detailed explanation)

This section lists **every control visible in the UI** and explains:

- Name
- Location
- Function
- Triggered actions (API calls / state changes)
- Expected result

4.0 Button & Control Mapping (VERY IMPORTANT)

This table maps each major control to:

- **Internal UI logic** (what frontend code runs)
- **API call** (what backend endpoint is hit)
- **Backend logic** (what simulator function executes)
- **What you see** (expected visual change)
- **Simulation / hardware effect**

Legend:

- UI logic: `./api/static/sim8051-app.js`
- API wrapper: `./api/static/sim8051-client.js`
- Backend routes: `./api/sandbox_api.py`
- Backend core: `./sim8051/session.py`

Button / Control	Action	Internal logic (frontend)	API call	Backend logic	Visual change	Hardware / simulation effect
Assemble (Assemble, #assemble)	Assemble editor source	<code>handleAssemble()</code>	<code>POST /api/v2/assemble</code>	<code>SimulatorSession.assemble()</code>	Assembler Output fills; editor execution mapping enabled	Loads ROM + listing; resets CPU state to program origin
Run (Run, #run)	Continuous run	<code>runLoop()</code>	<code>POST /api/v2/run</code> (repeated slices)	<code>SimulatorSession.run()</code>	Registers/PC/memory/trace update repeatedly	Executes instructions until stop reason (breakpoint/halt/timeout/etc.)
Step Into (Step Into, #step)	1 instruction	<code>handleSingleStep("step")</code>	<code>POST /api/v2/step</code>	<code>SimulatorSession.step()</code>	One trace entry; execution highlight moves	Executes one instruction + hardware tick
Step Over (Step Over, #step_over)	Step over calls	<code>handleSingleStep("stepOver")</code>	<code>POST /api/v2/step-over</code>	<code>SimulatorSession.step_over()</code>	May log multiple executed mnemonics	Runs until return address (or stop reason)
Step Out (Step Out, #step_out)	Run until return	<code>handleSingleStep("stepOut")</code>	<code>POST /api/v2/step-out</code>	<code>SimulatorSession.step_out()</code>	Call Stack updates	Runs until current function returns (or stop reason)
Step Back (Step Back, #step_back)	Reverse 1 step	<code>handleStepBack()</code>	<code>POST /api/v2/step-back</code>	<code>SimulatorSession.step_back()</code>	State reverts; trace re-rendered	Restores prior CPU/memory from bounded history
Pause (Pause, #pause)	Stop frontend run loop	inline handler sets paused and calls <code>refreshState()</code>	<code>GET /api/v2/state</code>	<code>snapshot()</code>	"Pause" flashes; state refresh	Stops issuing <code>/run</code> requests (execution stops advancing)
Stop (Stop, #stop)	Stop + reset	inline handler → <code>handleReset()</code>	<code>POST /api/v2/reset</code>	<code>SimulatorSession.reset()</code>	Console clears; PC/cycles reset	CPU reset to loaded program origin
Run To Cursor (Run To Cursor, #run_to_cursor)	Temporary breakpoint run	<code>handleRunToCursor()</code>	<code>POST /api/v2/breakpoints + run</code>	<code>set_breakpoints() + run()</code>	Breakpoint added temporarily	Runs until cursor-line PC is hit
Reset (Reset, #reset)	Reset CPU state	<code>handleReset()</code>	<code>POST /api/v2/reset</code>	<code>SimulatorSession.reset()</code>	PC/cycles reset	Restarts execution at origin
View (Code / Hardware)	Switch workspace	<code>setWorkspaceMode()</code>	none	none	Hardware board shown/hidden	Hardware view triggers a render + viewport fit
Architecture (#architecture_select)	Switch CPU model	<code>handleArchitectureChange()</code>	<code>POST /api/v2/architecture</code>	<code>SimulatorSession.set_architecture()</code>	Target chip label updates; default source loads	Rebuilds CPU+assembler; hardware resets for architecture
Endian (#endian_select)	ARM endian	<code>handleEndianChange()</code>	<code>POST /api/v2/endian</code>	<code>SimulatorSession.set_endian()</code>	Execution State endian updates	ARM loads/stores interpret multi-byte data as selected endian
Debug Mode (#debug_toggle)	Toggle debug	<code>handleDebugToggle()</code>	<code>POST /api/v2/debug</code>	<code>SimulatorSession.set_debug_mode()</code>	"Debug: On/Off" updates	Disables some compact fast paths; more detail, slower run
Execution (#execution_mode_select)	Realtime/Fast mode	<code>handleExecutionModeChange()</code>	<code>POST /api/v2/execution-mode</code>	<code>SimulatorSession.set_execution_mode()</code>	Execution field updates	Changes run pacing strategy
Speed (#run_speed)	Speed multiplier	<code>getRunSpeedMultiplier()</code> used by <code>runLoop()</code>	included in <code>POST /api/v2/run</code> payload	<code>run(... speed_multiplier)</code>	<code>Nx</code> label updates	Scales effective simulated Hz used for time
Breakpoint gutter	Toggle breakpoint	<code>Monaco.onMouseDown → toggleBreakpoint()</code>	<code>POST /api/v2/breakpoints</code> (after assembly)	<code>set_breakpoints()</code>	Red breakpoint glyph appears/disappears	Backend breaks when PC hits breakpoint PC(s)
Clear Breakpoints (#clear_breakpoints)	Remove all breakpoints	clears set → <code>syncBreakpoints()</code>	<code>POST /api/v2/breakpoints</code>	<code>set_breakpoints([])</code>	Breakpoint list becomes "None"	No breakpoint stops

Editor/ Control (#memory_edit_input)	Action/memory byte	Internal logic (frontend) handleMemoryEdit()	API call POST /api/v2/memory	Backend logic edit_memory()	Visual change State refresh; console log	Hardware / simulation effect Writes IRAM/SFR/XXRAM byte
Export State (#export_state)	Export session JSON	exportSessionState()	GET /api/v2/export	export_state()	File download	Saves CPU+program+hardware snapshot
Import State (#import_state)	Import session JSON	importSessionState()	POST /api/v2/import	import_state()	UI loads imported snapshot	Restores CPU+program+hardware snapshot
Download Output (#download_output)	Download snapshot JSON	downloadSnapshot()	none	none	File download	Saves current UI snapshot JSON
Waveform (#waveform_toggle)	Toggle drawer	setWaveDrawerOpen()	none	none	Drawer opens/closes	Shows recent signal activity from hardware logs
Theme (#theme_toggle)	Light/Dark	setTheme()	none	none	Theme colors change	UI-only
Help (#help_toggle)	Toggle popover	toggleAttribute("hidden")	none	none	Popover appears	UI-only
Hardware: add device (palette drag/drop)	Create component	createHardwareDeviceAt()	POST /api/v2/hardware/device + PATCH /api/v2/hardware/device	hardware.add_device() / update_device()	New device card appears	Adds virtual device, sets position/connections
Hardware: remove device (×)	Delete component	click handler on canvas	DELETE /api/v2/hardware/device	hardware.remove_device()	Device disappears	Removes virtual device and wiring
Hardware: connect pin↔device (drag wire)	Bind signal	_completeConnectionDrag()	PATCH /api/v2/hardware/device	hardware.update_device(... connections=...)	Wire appears	Connects MCU signal to device input/bus
Hardware: switch toggle	Drive input	handleHardwareSwitchToggle()	POST /api/v2/hardware/switch	hardware.set_switch_level()	Switch shows High/Low	Injects pin level into MCU input
Hardware: fault select	Inject fault	handleHardwareFaultChange()	POST /api/v2/hardware/fault	hardware.inject_fault()	Validation/Signals update	Simulates stuck/delay/noise faults on a signal
Hardware: save/load layout	Export/Import hardware	export/import handlers	GET /api/v2/hardware/export / POST /api/v2/hardware/import	hardware.export_state() / import_state()	File download / board updates	Saves/restores device state + wiring
Hardware: Run Hardware Test (#vh_test_btn)	Run validation suite	runHardwareValidationTests()	POST /api/v2/hardware/test	run_hardware_test()	Test results listed	Runs deterministic checks; reports PASS/FAIL

4.1 Run / debug controls (toolbar, left side)

Assemble

- **Location:** Toolbar, left group
- **Function:** Assemble the editor source into a program image (ROM + listing).
- **Triggered actions:** POST /api/v2/assemble with { code }
- **Expected result:**
 - Assembler Output table fills in (addresses + bytes)
 - Breakpoints are synced to backend PCs (if any were set)
 - Status shows success or assembly error (with line highlight)

Run

- **Location:** Toolbar, left group
- **Function:** Run continuously in slices until a stop reason occurs (breakpoint, halt, etc.).
- **Triggered actions:** repeated POST /api/v2/run with { max_steps: 100000, speed_multiplier }
- **Expected result:**
 - PC and cycles advance
 - Trace Timeline grows (summaries when many steps run)
 - Hardware state updates (if hardware is connected)
 - Stops automatically on breakpoints / halt / watchpoint (backend reason)

Step Into

- **Location:** Toolbar, left group
- **Function:** Execute exactly one instruction (source-level mapping shown via listing).
- **Triggered actions:** POST /api/v2/step
- **Expected result:** single instruction trace appears, PC moves, memory/register changes appear.

Step Over

- **Location:** Toolbar, left group
- **Function:** Execute "one statement", stepping over calls (runs until return address reached).
- **Triggered actions:** POST /api/v2/step-over
- **Expected result:** executes a small run slice; trace may show multiple steps.

Step Out

- **Location:** Toolbar, left group
- **Function:** Continue until the current function returns (call stack decreases).
- **Triggered actions:** POST /api/v2/step-out
- **Expected result:** executes a run slice until return; updates call stack and PC.

Step Back

- **Location:** Toolbar, left group
- **Function:** Reverse one step using bounded history ("time travel").
- **Triggered actions:** POST /api/v2/step-back
- **Expected result:**
 - If history exists: PC/registers/memory revert and trace resets appropriately.
 - If history empty: status shows "No reverse history."

Pause

- **Location:** Toolbar, left group
- **Function:** Stop the frontend run loop (no more /run calls).
- **Triggered actions:** No API call; frontend flips paused=true and fetches one refresh.
- **Expected result:** execution stops advancing; state is refreshed once from GET /api/v2/state.

Stop

- **Location:** Toolbar, left group (styled "danger")

- **Function:** Stop execution and reset the simulator (same effective outcome as Reset).
- **Triggered actions:** frontend calls Reset handler → `POST /api/v2/reset`
- **Expected result:** program returns to origin, cycles reset, console cleared.

Run To Cursor

- **Location:** Toolbar, left group
- **Function:** Run until the current editor cursor line is reached.
- **Triggered actions:**
 - Temporarily adds a breakpoint at the cursor line (mapped to PCs)
 - `POST /api/v2/breakpoints`
 - Runs (same as Run)
 - Restores the previous breakpoints afterward
- **Expected result:** execution stops when the cursor line's assembled address is hit.

Reset

- **Location:** Toolbar, left group (styled "danger")
- **Function:** Reload the current assembled program and reset CPU state.
- **Triggered actions:** `POST /api/v2/reset`
- **Expected result:** PC returns to program origin; cycles reset; hardware remains configured.

4.2 View + architecture controls (toolbar, right side)

View: Code / Hardware

- **Location:** Toolbar segmented group ("View")
- **Function:** Switch between the normal IDE view and the Virtual Hardware board.
- **Triggered actions:** frontend-only state change + re-render hardware
- **Expected result:**
 - Code: editor + standard panels visible
 - Hardware: Virtual Hardware panel visible and fitted to viewport

Architecture dropdown

- **Location:** Toolbar
- **Options:** 8051, ARM
- **Function:** Change the CPU + assembler model.
- **Triggered actions:** `POST /api/v2/architecture`
- **Expected result:**
 - Breakpoints cleared
 - A default sample program for the selected architecture is loaded into the editor
 - Endian control enables only for ARM

Endian dropdown

- **Location:** Toolbar
- **Options:** Little, Big
- **Function:** Control ARM data endianness in the ARM sandbox model.
- **Triggered actions:** `POST /api/v2/endian`
- **Expected result:** ARM memory loads/stores interpret multi-byte values using selected endian.

Debug Mode checkbox

- **Location:** Toolbar
- **Function:** Enable extra debug detail (can reduce run performance).
- **Triggered actions:** `POST /api/v2/debug` with `{ enabled }`
- **Expected result:** backend may disable "compact" execution paths and produce more detailed step data.

Execution dropdown

- **Location:** Toolbar
- **Options:** Realtime, Fast
- **Function:**
 - **Realtime:** tries to align simulated time with wall-clock time in short slices.
 - **Fast:** runs as quickly as backend limits allow (step budget/timeouts apply).
- **Triggered actions:** `POST /api/v2/execution-mode` with `{ mode }`
- **Expected result:** Run behaves differently (pacing vs throughput).

Speed slider + label

- **Location:** Toolbar
- **Range:** 1× to 10×
- **Function:** Multiply the effective clock rate used for simulated time.
- **Triggered actions:** frontend-only; value is sent with each `POST /api/v2/run` as `speed_multiplier`.
- **Expected result:** higher speed makes "Realtime" catch up faster and increases effective simulated Hz.

Clock Hz control

- **Location:** Toolbar
- **Presets:** 11.0592M, 12M, 16M, 24M, 48M, plus custom integer Hz.
- **Function:** Set the target oscillator/core clock used by cycle-to-time calculations.
- **Triggered actions:** `POST /api/v2/clock` with `{ hz }`.
- **Expected result:** 8051 hardware timing uses `clock_hz / 12` machine cycles per second; ARM timing uses the selected clock directly.

4.3 Base Converter (toolbar)

Purpose: Convert a number between Hex/Dec/Bin.

Controls:

- Convert-from dropdown (Hex|Dec|Bin)
- Input field ("Value")
- "to" Convert-to dropdown
- Output (read-only)
- Swap button

Triggered actions: frontend-only parsing and formatting **Expected result:** output updates as you type; "Invalid" on parse failure.

4.4 Export / import / output (toolbar)

Download Output

- **Function:** Downloads the current **full state snapshot JSON** to your computer.
- **Triggered actions:** frontend-only blob download (no API call).
- **Expected result:** `hexlog1c-<arch>-snapshot.json` is saved.

Export State

- **Function:** Export a portable session JSON (CPU + program + hardware layout/state).
- **Triggered actions:** `GET /api/v2/export`
- **Expected result:** `hexlog1c-<arch>-session.json` saved; can be imported later.

Import State

- **Function:** Import a previously exported session JSON.
- **Triggered actions:** `POST /api/v2/import`
- **Expected result:**

- Snapshot loads into UI (CPU state + program + hardware)
- Editor source updates to imported source
- Breakpoints cleared (you can add them again)

4.5 Waveform + help + theme (toolbar)

Waveform

- **Function:** Opens/closes the Waveform drawer.
- **Triggered actions:** frontend-only UI toggle.
- **Expected result:** Drawer shows detected toggling signals if available.

Waveform "Close"

- **Function:** Closes the Waveform drawer.

Help

- **Function:** Toggle a small contact popover.
- **Expected result:** shows email link.

Dark Mode

- **Function:** Toggles light/dark theme.
- **Triggered actions:** frontend-only; stored in `localStorage` as `sim-theme`.
- **Expected result:** UI colors change; Monaco theme changes if Monaco is loaded.

4.6 Breakpoints + memory edit (toolbar)

Clear Breakpoints

- **Function:** Remove all breakpoints.
- **Triggered actions:** `POST /api/v2/breakpoints` with empty list
- **Expected result:** breakpoint gutter markers disappear; Breakpoints summary becomes "None".

Memory Edit input

- **Placeholder:** Ex: `20H=12H` or `0x40=255`
- **Function:** Write a value directly into memory/SFR/XRAM based on the address.
- **Format:** `ADDRESS=VALUE`
 - Hex: `20H`, `0x20`
 - Bin: `0b1010`
 - Dec: `32`
- **Triggered actions:** `POST /api/v2/memory` with `{ space, address, value }`
 - 8051: `address < 0x00 → iram, address >= 0x00 → sfr`
 - ARM: writes go to `xram`
- **Expected result:** state updates; console logs the write.

4.7 Editor interactions (center pane)

Editor (Monaco)

- **Function:** write your program.
- **Breakpoint toggle:** click the **glyph margin** (left gutter) on a line.
- **Execution highlight:** current PC highlights the mapped source line after assembly.

Fallback behavior:

- If Monaco fails to load from CDN, a **textarea editor** is used automatically.

4.8 Assembler Output interactions (center pane)

Assembler Output shows:

- Row index
- Source line number
- Address
- Source text
- Bytes emitted

The "active row" highlights the instruction at the current PC.

4.9 Debugger Console (center pane)

Shows:

- "Assembled N statements"
- Instruction logs during stepping/running
- Error messages
- Run summaries (step counts, last instruction)

4.10 Trace Timeline (center pane)

Shows recent executed instructions:

- PC
- mnemonic
- cycles
- optional text and register diffs

4.11 Memory windows (right pane)

Displayed tables (read-only view; editing is via Memory Edit input):

- **Internal RAM** (`iram`)
- **XRAM Sample** (`xram_sample`, only a small window)
- **Code ROM** (`rom`)

4.12 Runtime Metrics (right pane)

Shows performance and telemetry such as:

- cycles, `clock_hz`
- steps per second
- UI timing measurements (receive-to-paint, round-trip, dropped frames)

4.13 Status bar (bottom)

- Left: human-readable status (including errors)
- Right: "Mode: ... | PC ... | Cycles ..."

4.14 Keyboard shortcuts

These shortcuts are handled by the frontend and work when you are not typing in a normal input field:

- **F5:** Run
- **F11:** Step Into

- **F10**: Step Over
- **Shift + F11**: Step Out
- **Shift + F10**: Step Back

5. Step-by-Step Usage Guide

5.1 How to start the system (local development)

1. Open a terminal in `./8051-Simulator`
2. Create and activate a Python virtual environment:
 - `python3 -m venv .venv`
 - `source .venv/bin/activate`
3. Install the project:
 - `pip install -e .`
4. Run the backend:
 - `flask --app api/index.py run --debug`
5. Open the UI:
 - `http://127.0.0.1:5000`

If Monaco fails to load (CDN blocked/offline), the app automatically falls back to a simple textarea editor.

5.2 Your first program (assemble + run)

1. In the **Editor**, type or paste an 8051 program (include `ORG` and `END`).
2. Click **Assemble**.
3. Check **Assembler Output** for the listing and bytes.
4. Click **Step Into** to execute one instruction at a time, or **Run** to execute continuously.
5. Watch:
 - **Execution State** (PC and cycles)
 - **Registers and Flags**
 - **Trace Timeline**
 - **Memory** tables

5.3 Breakpoints

1. Click the editor **left gutter** next to a line to toggle a breakpoint.
2. The Breakpoints summary (left pane) lists `<line>` entries.
3. Click **Run**.
4. The run stops when the PC reaches a breakpoint address.

Notes:

- Breakpoints are line-based in the UI, then translated to program counters using the assembled listing.
- Breakpoints only fully synchronize once the program has been assembled.

5.4 Run To Cursor

1. Click in the editor at the line you want to stop on.
2. Click **Run To Cursor**.
3. HexLogic creates a temporary breakpoint on that line, runs, then restores your original breakpoints.

5.5 Reverse stepping (Step Back)

1. Use Step Into / Run to generate history.
2. Click **Step Back** to revert the last executed instruction.

Limits:

- History is bounded (a fixed maximum depth).
- If you have not stepped/run yet, history is empty.

5.6 Using the base converter

1. Pick input base (Hex/Dec/Bin).
2. Type a number.
3. Pick output base.
4. Use **Swap** to flip directions and keep working.

5.7 Export / import a session (recommended for labs)

To save:

- Click **Export State** and keep the downloaded JSON.

To restore:

- Click **Import State** and select the previously exported JSON.

6. Hardware Interaction Guide

HexLogic includes a Virtual Hardware board to visualize and test GPIO-driven programs.

6.1 Entering hardware mode

1. In the toolbar, under **View**, click **Hardware**.
2. The Virtual Hardware board becomes visible.

6.2 Available MCU pins (signals)

For **8051**, available pins are:

- `P0.0 ... P0.7`
- `P1.0 ... P1.7`
- `P2.0 ... P2.7`
- `P3.0 ... P3.7`

8051 special notes (important for realistic IO programs):

- **P0** is marked as **open-drain** in the model metadata (electrical behavior is simplified; see Known Issues).
- **External interrupts**: `P3.2` (INT0) and `P3.3` (INT1) are wired into the interrupt latch logic when you drive them via Virtual Hardware or pin injection.
- **Timer counter inputs**: `P3.4` (T0) and `P3.5` (T1) are used for counter-mode edge counting.
- **Timer 2 pins (subset model)**: `P1.0` and `P1.1` are monitored for Timer2 edge inputs in the core.

For bus connections:

- 8-bit buses: `P0`, `P1`, `P2`, `P3`
- 4-bit buses: `P0_LOW`, `P0_HIGH`, ..., `P3_LOW`, `P3_HIGH`

For **ARM**, available pins are:

- `GPIOA.0 ... GPIOA.15`
- Buses like `GPIOA_LOW`, `GPIOA_HIGH`, and 4-bit groups.

6.3 Adding hardware components

Use the **Components palette** (left sidebar in hardware view):

- LED
- Switch
- LED Array
- 7-Segment
- Stepper

How to add:

1. Click and drag a component from the palette.
2. Drop it onto the board area.

6.4 Connecting MCU pins to devices (wiring)

To connect:

1. Drag from an MCU pin (a small node labeled like `P1.0`)...
2. ...to a device connection node on a device card...
3. Release to create a wire.

Connection rules:

- A device connection can be a single **pin** (e.g., LED, Switch).
- Or a **bus** (e.g., LED Array uses an 8-bit bus; Stepper uses a 4-bit bus).

6.5 Moving, selecting, and navigating the board

- **Move a device:** drag it on the board.
- **Select a device:** click its card.
- **Multi-select:** drag a selection rectangle on empty space.
- **Additive selection:** hold **Ctrl** (or **Cmd** on macOS) while selecting.
- **Pan:** hold **Alt** and drag on empty space.
- **Zoom:** mouse wheel over the board (or use zoom buttons).

6.6 Using each device type (expected behavior)

LED (single pin)

- **Connect:** `pin` → e.g., `P1.0`
- **Expected behavior:** LED is ON when the pin level is high; OFF when low.

Switch (single pin input driver)

- **Connect:** `pin` → e.g., `P3.2`
- **Use:** click the switch checkbox on the device card ("High").
- **Expected behavior:** the switch injects a 0/1 level into the MCU pin.

Important:

- The firmware must **read** that pin/port to observe it.
- Hardware validation may warn: "switch toggled but firmware has not sampled the input".

LED Array (8-bit bus)

- **Connect:** `bus` → e.g., `P1`
- **Expected behavior:** each bit drives one LED; the widget also shows the bus value.

7-Segment (8-bit bus)

- **Connect:** `Segment Bus` → e.g., `P2`
- **Expected behavior:** displays a digit for common 7-segment encodings.

Stepper (4-bit bus)

- **Connect:** `Coil Bus` → e.g., `P1_LOW`
- **Expected behavior:** rotates when a valid stepping sequence is observed.

6.7 Hardware debug panel (bottom of hardware view)

Port Monitor

- Shows port summaries (name, hex, binary) reported by the backend hardware model.

Component State

- Lists components; click an entry to inspect that component.

Validation

- PASS / WARN / ERROR issues detected by the backend hardware validator.
- Use this to detect missing wiring, too-fast toggling, invalid sequences, etc.

Signal Log

- Recent transitions (pin name, value, time_ms).

Logic Analyzer

- A multi-signal waveform view based on recent `Signal Log` entries.
- Mouse wheel: zoom; Shift+wheel: pan time window; move cursor to inspect time.

Inspector

- Shows selected component details, metrics, connections, and signal health.
- Includes **Fault injection** controls per connected signal:
 - `stuck_high`, `stuck_low`, `delay`, `noise`, or `none`

Test Mode

- Click **Run Hardware Test** (top of hardware view) to run deterministic backend checks.
- Results appear here as PASS/FAIL lines.

7. Troubleshooting Guide

7.1 "Backend API unreachable"

Symptoms:

- Red error panel saying backend is unreachable
- Assemble/Run/Step do nothing

What it means:

- The UI cannot reach `/api/v2/*`.

Fix checklist:

- Start the Flask backend locally and reload the page.
- If deployed behind a CDN, ensure the proxy routes `/api/v2/*` to the backend service.

7.2 Assembly failed (error highlight)

Symptoms:

- Status bar shows an error
- Error box appears, often with a source line number

Fix:

- Check syntax (labels, commas, immediate formats like `#01H`, correct `ORG` / `END`).
- Assemble again.

7.3 Breakpoints don't trigger

Common causes:

- You didn't click **Assemble** after editing (PC mapping comes from the listing).
- You set a breakpoint on a non-executable line (comment/blank).

Fix:

- Assemble again and ensure the line appears in Assembler Output mapping.

7.4 Hardware switch doesn't affect the program

Common causes:

- Firmware never reads the port bit (no sampling).
- The pin is configured/used as output in firmware while the switch is driving it.

Fix:

- Add a port read in firmware (e.g., `MOV A,P3` then test bits).
- Check Validation panel warnings for "firmware has not sampled the input".

7.5 LED doesn't blink (timing confusion)

Common causes:

- Your delay loop is too fast (blinks faster than human-visible threshold).
- Execution mode is Fast (time is not paced to wall clock).
- Speed multiplier is too high.

Fix:

- Use **Execution = Realtime** and **Speed = 1×**.
- Use timers for accurate periods.
- Watch Validation warnings: "toggling too fast".

7.6 "No waveform channels detected"

Waveform drawer depends on observed signal activity (from the Virtual Hardware signal log).

Fix:

- Connect a hardware device to a pin you toggle.
- Run/step the program enough to produce transitions.
- Check Signal Log in hardware view.

8. Known Issues (if any)

These were detected by code-level inspection of the current codebase.

8.1 SFR (Special Function Register) table is not visible in the UI

- **Observed behavior:** UI renders IRAM, XRAM sample, and Code ROM, but does not render the `sfr` dump.
- **Why it matters:** users can't directly see port SFR values (`P0`, `P1`, `P2`, `P3`, `TCON`, `SCON`, etc.) in the memory tables.
- **Likely cause:** `frontend.renderMemory()` doesn't render `snapshot.sfr`.
- **Recommendation:** add an "SFR" memory table (space= `sfr`) and include it in the right-pane memory panel.

8.2 Watchpoints exist in the backend API, but there is no UI to manage them

- **Observed behavior:** backend route `POST /api/v2/watchpoints` exists, but there are no watchpoint controls in the UI.
- **Recommendation:** add a Watchpoints panel/editor and sync it to the backend.

8.3 Serial RX injection exists in the backend API, but there is no UI control

- **Observed behavior:** backend route `POST /api/v2/serial/rx` exists, but the UI doesn't expose it.
- **Recommendation:** add a Serial panel (RX inject + TX log).

8.4 Target Explorer is informational only (no real file/project loading)

- **Observed behavior:** the tree is a static placeholder (shows `main.asm`, `trace`, etc.) and isn't interactive.
- **Recommendation:** add "Open example", "Load file", and "Save" capabilities.

8.6 Stop and Reset are effectively the same action

- **Observed behavior:** Stop triggers a reset; Reset also resets.
- **Recommendation:** either change Stop to "Stop (no reset)" or rename to "Stop & Reset".

8.7 8051 P0 open-drain is exposed as metadata but not fully enforced

- **Observed behavior:** P0 is marked `open_drain` in metadata, but the electrical behavior is simplified.
- **Recommendation:** model P0 high as floating unless an external pull-up is present (for closer realism).

9. Best Practices

- **Assemble early and often:** source→PC mapping and breakpoints depend on the assembled listing.
- **Use Realtime + 1×** when learning timing: it's the closest to "real time" in the UI.
- **Keep Debug Mode off unless needed:** it can reduce run speed.
- **Use Virtual Hardware for IO programs:** connect LEDs/switches to confirm pin behavior.
- **Read inputs explicitly:** if you toggle a switch, your firmware must read the port/pin to observe it.
- **Export State for checkpoints:** export a working session before making large edits.
- **Start with small programs:** validate instruction effects before adding loops/interrupts.

Optional: UX improvement suggestions (nice-to-have)

- Rename **Download Output** → **Download Snapshot** (it downloads state JSON).
- Add an explicit **SFR panel** and a **Port view** (P0–P3 bits) in Code workspace.

- Add a **Watchpoints** UI and a **Serial console** panel.
- Add an **Examples dropdown** (load files from `./examples/`).
- Add an on-screen **keyboard shortcuts cheat sheet** (F5/F10/F11).